

Heap

COMP211 Connor McMahon

Quiz 2 Reminders

- You can find all quiz 2 information here
  - https://uncch.instructure.com/courses/120642/pages/quiz-2
- TA-led Review Session in FB141 (TODAY)
  - Monday, October 12 5-6:30pm
- Tutoring in the CSXL
  - Monday, October 5, 5-6pm
  - Thursday, October 8, 5-6pm
  - Tuesday, October 13 5-6pm
- TAs are also available to help you prepare during office hours!
  - Monday-Thursday: 11:30am-5pm
  - Friday: 11:30am-3:30pm

array_of_strings.c

- This program builds a dynamically growing array of strings.
  - It repeatedly asks the user to enter words.
  - Each word is stored on the heap.
  - The program keeps the words in a dynamically resized array of char * pointers.
  - If the array fills up, it doubles its capacity using realloc.
  - When the user types "quit", the program:
    - Prints all stored words.
    - Frees all allocated memory.



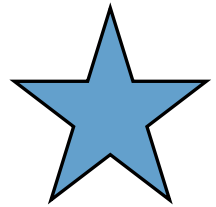
```
/* TODO: Allocate memory for an array of char*
pointers with
INITIAL_CAPACITY elements. Each element will
eventually point to a
dynamically allocated string. */
words = malloc(capacity * sizeof(char*));
if (!words) {
   fprintf(stderr, "malloc failed");
   return 1;
}
```



```
/* TODO: Use realloc to resize the existing array of
char* pointers
to hold the new (doubled) capacity. Do NOT lose the
original
pointer if realloc fails. */
char** temp = realloc(words, capacity * sizeof(char*));
if (!temp) {
   fprintf(stderr, "realloc failed");
   break;
}
words = temp;
```



```
/* TODO: Allocate enough space on the heap to
store the buffer string in
the heap, including space for the null
terminator ('\0'). */
words[count] = malloc((strlen(buffer) + 1) *
sizeof(char));
if (!words[count]) {
   fprintf(stderr, "malloc failed");
   break;
}
```



```
printf("\nYou entered:\n");
for (size_t i = 0; i < count; i++) {
   printf("%zu: %s\n", i, words[i]);
   /* TODO: Free the dynamically allocated
   string
   to prevent a memory leak. */
   free(words[i]);
}

/* TODO: After freeing each individual
string, free the array
that stores the char* pointers. */
free(words);
```

Garbage Collection in Java

The Java Virtual Machine

- The **Java Virtual Machine (JVM)** is the software that runs Java programs.

- It takes compiled **Java bytecode** and executes it on the computer.

The Java Virtual Machine

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

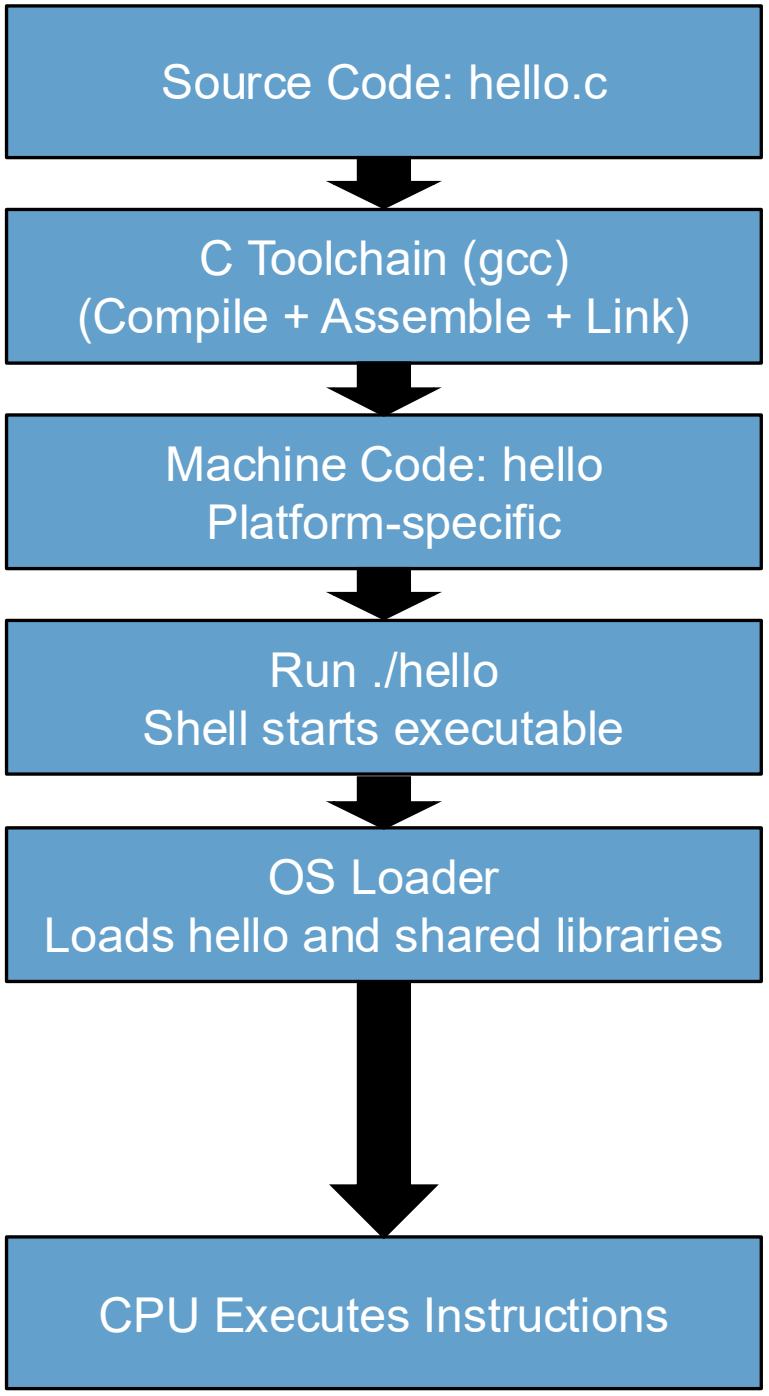
First, compile the source code: `$ javac Hello.java`

This creates `Hello.class`, which contains Java bytecode.

Then run the program: `$ java Hello`

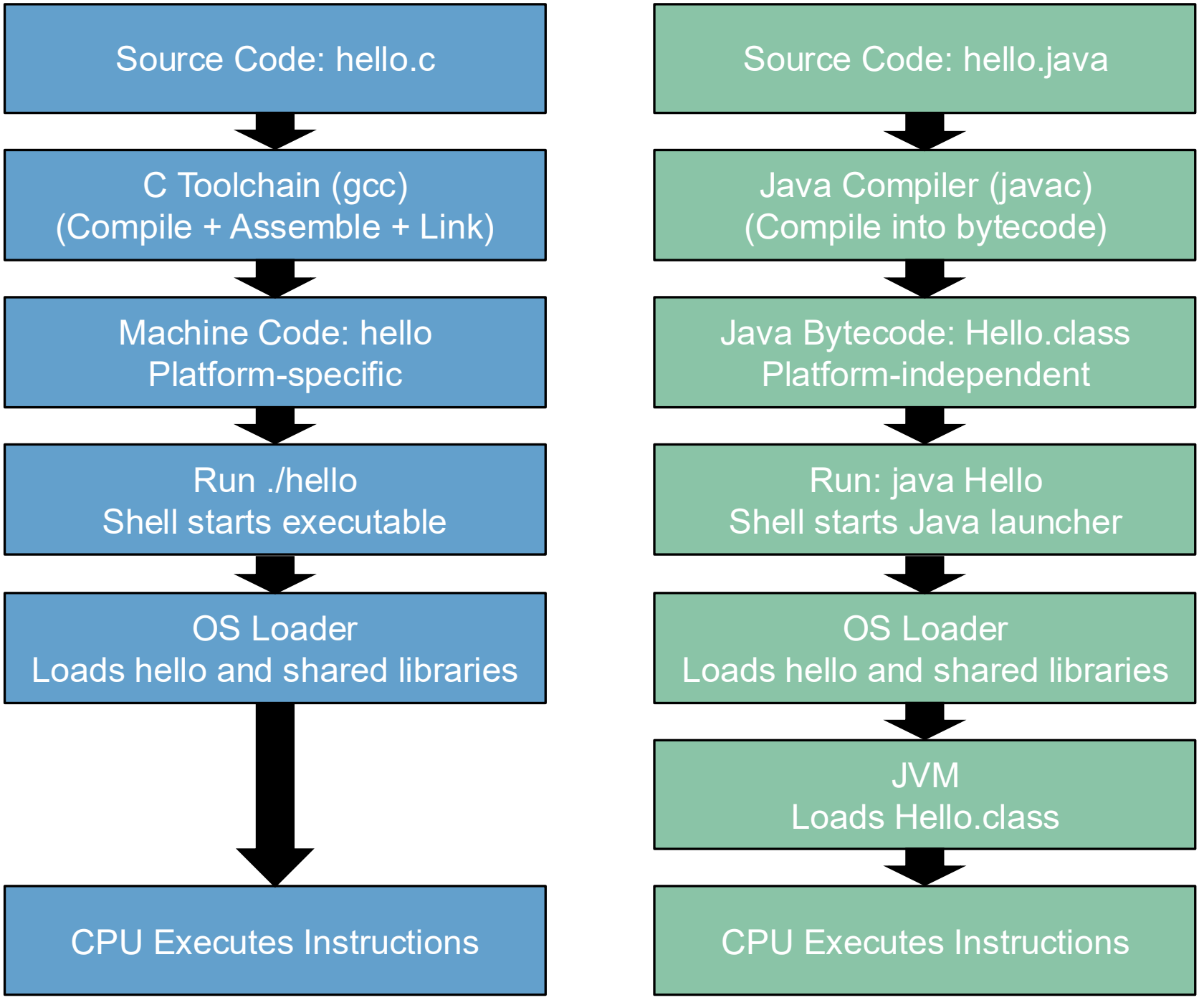
The java command launches the JVM, which loads and executes the bytecode.

Output: `Hello, World!`


Source Code: hello.c
C Toolchain (gcc)
(Compile + Assemble + Link)
Machine Code: hello
Platform-specific
Run ./hello
Shell starts executable
OS Loader
Loads hello and shared libraries
CPU Executes Instructions

Platform-dependent vs Platform-independent

- **Platform-dependent:** Code or instructions that are designed to run on a particular type of computer architecture (such as x86-64 or ARM).
  - Machine-specific code may need to be modified or recompiled to run on a different architecture.
  - *Example:* A C program compiled into x86-64 machine code cannot run directly on an ARM processor.
- **Platform-independent:** Code that can run on different types of computers or operating systems without needing to be rewritten or recompiled for each platform.
  - *Example:* Java bytecode is platform-independent because the same .class file can run on different systems, as long as each system has a compatible Java Virtual Machine (JVM).


Source Code: hello.c
C Toolchain (gcc)
(Compile + Assemble + Link)
Machine Code: hello
Platform-specific
Run ./hello
Shell starts executable
OS Loader
Loads hello and shared libraries
CPU Executes Instructions
Source Code: hello.java
Java Compiler (javac)
(Compile into bytecode)
Java Bytecode: Hello.class
Platform-independent
Run: java Hello
Shell starts Java launcher
OS Loader
Loads hello and shared libraries
JVM
Loads Hello.class
CPU Executes Instructions

JVM Benefits

- Portability
  - The same compiled Java program can run on different operating systems without being recompiled, as long as a compatible JVM is available.

- Memory management:
  - The JVM automatically allocates memory for objects and uses garbage collection to reclaim memory that is no longer needed.

- Performance optimization:
  - The JVM can identify frequently executed code and compile it into optimized machine code to improve performance.

Memory Management

- A big difference between Java and C is who is responsible for managing memory.

- In C, the programmer explicitly allocates and frees dynamically allocated memory.

- In Java, the programmer creates objects, and the JVM automatically reclaims memory when those objects become unreachable.

Allocating and Freeing Memory

C

```
int *p = malloc(sizeof(int));

*p = 42;

printf("%d\n", *p);

free(p);
p = NULL;
```

The programmer explicitly allocates and releases the memory.

In C, forgetting free(p) can cause a memory leak.

Java

```
int[] p = new int[1];

p[0] = 42;

System.out.println(p[0]);

p = null;
```

The JVM reclaims the array when it becomes unreachable. Setting p to null is not normally necessary.

Property	C	Java
Memory allocation	Automatic for local variables; explicit `malloc()` for dynamic memory	Automatic
Memory reclamation	Explicit `free()` for dynamically allocated memory; automatic for local variables	Automatic garbage collection for unreachable objects
Use-after-free	Possible	Prevented for ordinary managed objects
Double-free	Possible	Not applicable to managed objects
Memory leaks	Possible	Still possible if unused objects remain reachable

The Garbage Collector

- **Garbage Collector** (GC): A component of a program's runtime environment that automatically identifies and reclaims memory occupied by objects that are no longer needed, allowing that memory to be reused.
- In Java, the garbage collector identifies objects that are no longer reachable by the running program and reclaims their memory.

```
Person p = new Person("Alice");
p = null;
```

- After p = null, the Person object is no longer reachable (assuming no other references to it exist), so it becomes eligible for garbage collection.

The Garbage Collector can slow down the program

- Garbage collection (GC) can slow down a Java program because the JVM must spend time finding and removing objects that are no longer needed.

- This can affect performance in two ways:

  - Pausing the program: Some garbage collection operations temporarily stop the program while the JVM manages memory. These are called stop-the-world pauses.

  - Using CPU resources: Garbage collection requires processing power to find unused objects and reclaim their memory. This competes with the program for CPU time.

Reference Counting

- Reference counting is one of the simplest approaches to automatic memory management.
- The idea is to maintain a count of how many references point to each object.

```
Person a = new Person("Alice");  // 1 reference to Alice
Person b = a;                    // 2 references
a = null;                        // 1 reference
b = null;                        // 0 references
```

- When the reference count reaches zero, the object becomes eligible for garbage collection. Java does not necessarily reclaim its memory immediately.

The Problem with Reference Counting

- Circular References

```
Node a = new Node();  // Node A: 1 reference (a)
Node b = new Node();  // Node A: 1, Node B: 1 (b)

a.next = b;           // Node A: 1, Node B: 2 (b, a.next)
b.next = a;           // Node A: 2 (a, b.next), Node B: 2

a = null;             // Node A: 1 (b.next), Node B: 2
b = null;             // Node A: 1, Node B: 1
```

- The objects still reference each other, so their reference counts are nonzero even though neither can be accessed by the program.
- Simple reference counting cannot reclaim this cycle.
- This is a major reason Java uses tracing garbage collectors instead.

Mark-and-sweep

- Mark-and-sweep is a tracing garbage collection algorithm.

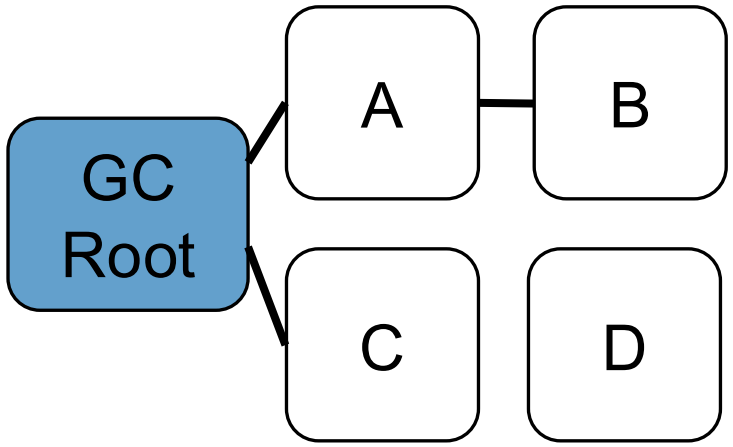
- A GC root is a starting point that Java's garbage collector uses to find objects that are still in use.

- It works in two phases:
  - Mark: Starting from GC roots, traverse references and mark every reachable object.
  - Sweep: Examine allocated objects and reclaim those that were not marked.

Mark-and-sweep

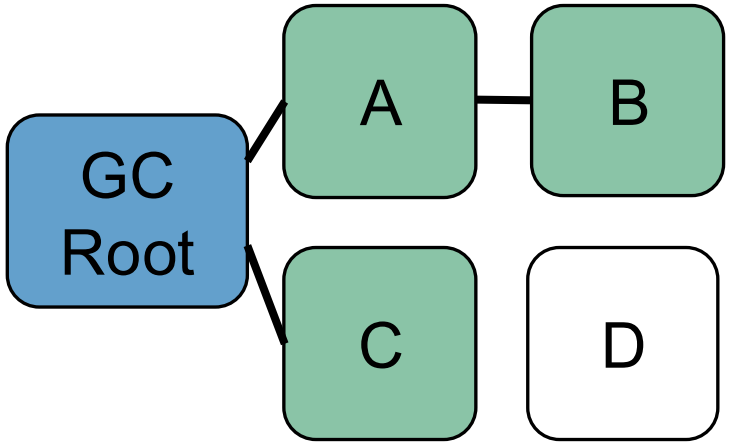
- Mark: Starting from GC roots, traverse references and mark every reachable object.
- Sweep: Examine allocated objects and reclaim those that were not marked.

Initial



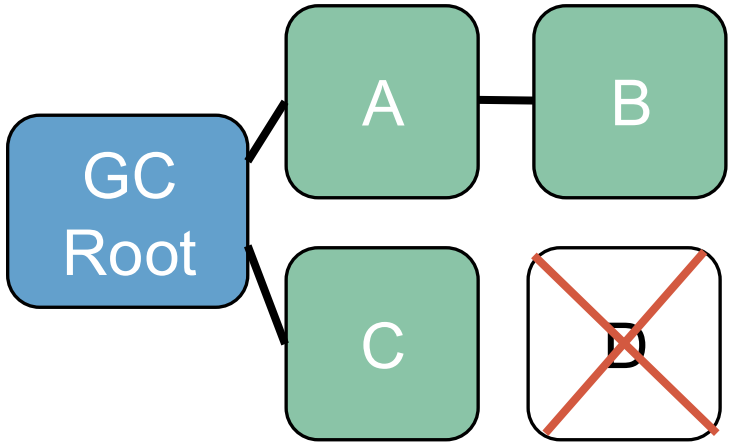
Object Graph. Four allocated objects exist.

Mark



The collector traverses the object graph and marks A, B, and C as reachable. D remains unreachable.

Sweep



The collector reclaims D because it was not marked.

Memory Management: C vs. Java

- C gives programmers more direct control over dynamically allocated memory, but also makes them responsible for freeing it correctly.

- Java automatically manages object memory, reducing common memory errors at the cost of some runtime overhead.
  - Garbage collection automates memory reclamation, reducing the need for programmers to manually free dynamically allocated memory. However, memory is not necessarily reclaimed immediately when an object becomes unreachable.

Quiz Review

Fill in the low-level memory diagram for the following program **after all statements have executed**. Assume that the stack frame for main begins at address 0xFFFFAAAA0040,

```
int main() {
    int64_t arr[] = {5, 10, 15, 20, 25};
    int64_t* p = &arr[2];

    *p = 50;
    p[-1] = 100;
    *(p + 2) = arr[0] + arr[1];
    p = p - 2;
    arr[3] = *(p + 2) + p[1];
    arr[0] = &arr[4] - p;
}
```

Address	Contents	Variable
0xFFFFAAAA0040		
0xFFFFAAAA0038		
0xFFFFAAAA0030		
0xFFFFAAAA0028		
0xFFFFAAAA0020		
0xFFFFAAAA0018		
0xFFFFAAAA0010		
0xFFFFAAAA0008		
0xFFFFAAAA0000		
0xFFFFAAA9FFF8		
0xFFFFAAA9FFF0		

Fill in the low-level memory diagram for the following program **after all statements have executed**. Assume that the stack frame for main begins at address 0xFFFFAAAA0040,



```
int main() {
    int64_t arr[] = {5, 10, 15, 20, 25};
    int64_t* p = &arr[2];

    *p = 50;
    p[-1] = 100;
    *(p + 2) = arr[0] + arr[1];
    p = p - 2;
    arr[3] = *(p + 2) + p[1];
    arr[0] = &arr[4] - p;
}
```

Address	Contents	Variable
0xFFFFAAAA0040	0	RA
0xFFFFAAAA0038	~~25~~ 105	arr[4]
0xFFFFAAAA0030	~~20~~ 150	arr[3]
0xFFFFAAAA0028	~~15~~ 50	arr[2]
0xFFFFAAAA0020	~~10~~ 100	arr[1]
0xFFFFAAAA0018	~~5~~ 4	arr[0]
0xFFFFAAAA0010	~~0xFFFFAAAA0028~~ 0xFFFFAAAA0018	p
0xFFFFAAAA0008	(unused)	
0xFFFFAAAA0000	(unused)	
0xFFFFAAA9FFF8	(unused)	
0xFFFFAAA9FFF0	(unused)	

```
int main() {
    int64_t arr[] = {5, 10, 15, 20, 25};
    int64_t* p = &arr[2];

    *p = 50;
    p[-1] = 100;
    *(p + 2) = arr[0] + arr[1];
    p = p - 2;
    arr[3] = *(p + 2) + p[1];
    arr[0] = &arr[4] - p;
}
```

Address	Contents (4 bytes)	Variable
0xFFFFAAAA0044		
0xFFFFAAAA0040		
0xFFFFAAAA003C		
0xFFFFAAAA0038		
0xFFFFAAAA0034		
0xFFFFAAAA0030		
0xFFFFAAAA002C		
0xFFFFAAAA0028		
0xFFFFAAAA0024		
0xFFFFAAAA0020		
0xFFFFAAAA001C		
0xFFFFAAAA0018		
0xFFFFAAAA0014		
0xFFFFAAAA0010		
0xFFFFAAAA000C		
0xFFFFAAAA0008		
0xFFFFAAAA0004		
0xFFFFAAAA0000		



```
int main() {
    int64_t arr[] = {5, 10, 15, 20, 25};
    int64_t* p = &arr[2];

    *p = 50;
    p[-1] = 100;
    *(p + 2) = arr[0] + arr[1];
    p = p - 2;
    arr[3] = *(p + 2) + p[1];
    arr[0] = &arr[4] - p;
}
```

Address	Contents (4 bytes)	Variable
0xFFFFAAAA0044	0	RA (upper 4 bytes)
0xFFFFAAAA0040	0	RA (lower 4 bytes)
0xFFFFAAAA003C	0	arr[4] (upper 4 bytes)
0xFFFFAAAA0038	~~25~~ 105	arr[4] (lower 4 bytes)
0xFFFFAAAA0034	0	arr[3] (upper 4 bytes)
0xFFFFAAAA0030	~~20~~ 150	arr[3] (lower 4 bytes)
0xFFFFAAAA002C	0	arr[2] (upper 4 bytes)
0xFFFFAAAA0028	~~15~~ 50	arr[2] (lower 4 bytes)
0xFFFFAAAA0024	0	arr[1] (upper 4 bytes)
0xFFFFAAAA0020	~~10~~ 100	arr[1] (lower 4 bytes)
0xFFFFAAAA001C	0	arr[0] (upper 4 bytes)
0xFFFFAAAA0018	~~5~~ 4	arr[0] (lower 4 bytes)
0xFFFFAAAA0014	0x0000FFFF	p (upper 4 bytes)
0xFFFFAAAA0010	~~0xAAAA0028~~ 0xAAAA0018	p (lower 4 bytes)
0xFFFFAAAA000C	(unused)	
0xFFFFAAAA0008	(unused)	
0xFFFFAAAA0004	(unused)	
0xFFFFAAAA0000	(unused)	